ADR-002: データベース — Supabase をプライマリDBとして採用

ステータス

承認

日付

2026-03-12

コンテキスト

シャチョツーは **マルチテナントSaaS** である。1つのPostgreSQLクラスターに複数企業（テナント）のデータが共存するため、テナント間のデータ分離はプロダクトの根幹的なセキュリティ要件になる。

必要なDB機能

要件	重要度	詳細
リレーショナルDB (PostgreSQL)	必須	ナレッジ・フロー・ルール等の構造化データ
ベクトル検索 (pgvector)	必須	Q&A検索・類似ナレッジ・セマンティックキャッシュ
行レベルセキュリティ (RLS)	必須	`company_id` ベースのテナント分離
認証・セッション管理	必須	JWTトークン発行・ユーザー管理
ファイルストレージ	重要	音声ファイル・ドキュメント・帳票の保管
リアルタイム購読	準重要	フロントエンドへの能動提案プッシュ
バックアップ・リカバリ	必須	企業データの保全
マネージドサービス	重要	1人開発チームでのインフラ管理負担最小化

技術的背景

- **テナント分離方式の選択肢**: (a) DB/スキーマ分離、(b) RLSによる行レベル分離、(c) アプリケーション層フィルタリング

- (a) はテナント数増加でコスト爆発。(c) はコードバグによるデータ漏洩リスクが高い。(b) が最もコスト効率とセキュリティのバランスが良い
- **ベクトル次元**: Voyage AI voyage-3 は1024次元ベクトルを出力。HNSW インデックスで高速近傍探索が必要
- **スケール想定**: Phase 0-3（〜3,000社）はマネージドサービスで対応。その後は自社運用PostgreSQLへの移行を評価

検討した選択肢

選択肢A: Supabase — 採用案

概要: PostgreSQLベースのBaaSプラットフォーム。OSS版とクラウド版がある。

メリット:
- **RLS ネイティブサポート**: SupabaseはRLSをファーストクラスで扱う。ダッシュボードからRLSポリシーを定義・テストでき、マルチテナント実装の標準パターンが充実している

`sql -- Supabaseで推奨されるRLSパターン CREATE POLICY "company_isolation" ON knowledge_items USING (company_id = auth.jwt() ->> 'company_id');`

- **pgvector 統合**: `pgvector` 拡張がデフォルトで有効化済み。HNSWインデックスもサポート

`sql CREATE INDEX ON knowledge_items USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);`

- **認証 (Supabase Auth)**: JWT発行・リフレッシュ・ソーシャルログイン・MFAが組み込み。独自の認証サーバーを実装・運用する必要がない
- **ストレージ (Supabase Storage)**: S3互換。RLSポリシーをストレージバケットにも適用できる。音声ファイル・帳票PDFの保管に使用
- **リアルタイム**: PostgreSQLのWALをリッスンし、テーブル変更をWebSocketでクライアントにプッシュ。能動提案のリアルタイム表示に活用
- **マネージド**: バックアップ・スケーリング・パッチ適用をSupabaseが担う。1人チームのインフラ負担を大幅削減
- **OSS**: コアのSupabaseはApache 2.0ライセンス。将来のセルフホスト移行パスが存在する
- **コスト**: Proプラン $25/月〜。競合マネージドPostgreSQL（Aurora Serverless等）より大幅に安価。pgvector・Auth・Storageが全て含まれる

デメリット:
- **ベンダーロックイン**: SupabaseのAuthスキーマ（`auth.users`）やStorage APIはSupabase固有。Supabase Authからの移行は一定の工数が必要
- **スケール上限**: 単一インスタンスの限界がある（数万テナント規模では性能問題が出る可能性）。Supabaseの公式には〜10万ユーザーまで対応とあるが、実績は要確認
- **地理的レイテンシ**: データセンターが東京リージョンにあること要確認（2026年時点: ap-northeast-1

東京リージョン利用可能）
- **SQL管理**: Supabaseダッシュボード経由のスキーマ変更はバージョン管理が難しい → `db/migrations/` でSQLファイル管理することで対応

選択肢B: PlanetScale (MySQL互換) — 却下

概要: Vitessベースのサーバーレスデータベース。ブランチ機能が特徴。

メリット:
- ブランチ機能でスキーマ変更を安全に実施できる
- スケーラビリティが高い

デメリット:
- **RLS非対応**: MySQLはPostgreSQLのようなRLSをネイティブサポートしない。アプリケーション層でのテナント分離が必要 → セキュリティリスク
- **pgvector非対応**: ベクトル検索のために別サービス（Pinecone等）が必要 → コスト増・アーキテクチャ複雑化
- **PostgreSQL非互換**: SQLの方言差異、関数の違い等で移行・学習コストが発生

→ **決定**: RLS非対応とpgvector非対応の時点で要件を満たさない。却下。

選択肢C: Neon (PostgreSQL サーバーレス) — 却下

概要: サーバーレスPostgreSQLのホスティングサービス。コールドスタートが特徴。

メリット:
- サーバーレスでスケールゼロが可能（開発時のコスト最小化）
- PostgreSQL互換性が高い
- pgvector サポートあり

デメリット:
- **認証なし**: 独自の認証サーバーを別途構築・運用が必要
- **ストレージなし**: 音声・ドキュメントファイルは別サービス（S3等）が必要
- **RLSはPostgreSQL標準**: 実装は可能だが、Supabaseのようなダッシュボードサポートなし。設定ミスによるRLSバイパスリスクが高い
- **コールドスタート**: 頻繁なAPIリクエストがある本番環境では予期しないレイテンシスパイク
- **マルチテナント実績**: Supabaseよりマルチテナント向けのドキュメント・実例が少ない

→ **決定**: 認証・ストレージの別途調達コストと、RLS実装支援の薄さが問題。Supabase比で総合的に劣る。却下。

選択肢D: AWS Aurora Serverless v2 (PostgreSQL互換) + Cognito + S3 — 却下

概要: AWSのフルマネージドPostgreSQL。エンタープライズ向けの実績が豊富。

メリット:
- **エンタープライズ実績**: 大規模SaaSの本番実績が豊富
- **スケーラビリティ**: 無制限（コスト次第）
- **コンプライアンス**: SOC 2, PCI DSS, HIPAA等の認証が豊富
- **pgvector サポート**: Aurora PostgreSQL 15.2以降でサポート

デメリット:
- **コスト**: Aurora Serverless v2 は最低でも0.5 ACU常時稼働 ≈ $50/月〜。RDS Proxyを加えると$100/月〜。Supabase Proの4倍以上のコスト（初期フェーズでは許容しがたい）
- **認証の別途構築**: Amazon Cognitoは設定が複雑。JWT + RLSの連携に相当な実装工数がかかる
- **GCPと別クラウド**: インフラはGCP（Cloud Run）を採用しているため、AWS利用でマルチクラウド管理コストが増大
- **学習コスト**: Aurora + Cognito + S3 + VPC設定の組み合わせは、1人チームには過大な運用負担

→ **決定**: コストと1人チームの運用負担が許容範囲を超える。Phase 3以降の移行先として将来評価する。

選択肢E: セルフホスト PostgreSQL (GCP Cloud SQL / GKE) — 将来候補

概要: GCP Cloud SQL（マネージドPostgreSQL）またはGKEでPostgreSQLを自己運用。

メリット:
- **完全コントロール**: 設定・拡張・スケーリング戦略を自由に決定
- **コスト最適化**: 大規模では従量課金より固定コストの方が安い可能性
- **データ主権**: データが完全に自社GCPプロジェクト内に収まる

デメリット:
- **運用負担**: バックアップ・フェイルオーバー・パッチ適用・モニタリングを全て自社で担う
- **認証・ストレージ**: 別途実装が必要
- **開発速度**: インフラ構築だけでMVPが遅延する

→ **決定**: Phase 3〜4（3,000社超、月次AWS/GCPコスト¥500万超が目安）で移行を評価する。それ以前は却下。

決定

Supabase を Phase 0〜3（〜3,000社規模）のプライマリデータベースとして採用する。

採用スコープ:
- **Supabase PostgreSQL**: 全テーブル（RLS必須）
- **Supabase pgvector**: ナレッジ埋め込み・セマンティックキャッシュ
- **Supabase Auth**: ユーザー認証・JWT発行
- **Supabase Storage**: 音声ファイル・ドキュメント・帳票
- **Supabase Realtime**: 能動提案のリアルタイムプッシュ

RLSの実装原則

全テーブルに以下を必須適用する（例外なし）:

```
-- 1. RLS有効化
ALTER TABLE {table_name} ENABLE ROW LEVEL SECURITY;

-- 2. SELECTポリシー（自社データのみ閲覧可能）
CREATE POLICY "{table_name}_select_policy" ON {table_name}
  FOR SELECT USING (
    company_id = (auth.jwt() ->> 'company_id')::uuid
  );

-- 3. INSERT/UPDATE/DELETEも同様に制限
CREATE POLICY "{table_name}_insert_policy" ON {table_name}
  FOR INSERT WITH CHECK (
    company_id = (auth.jwt() ->> 'company_id')::uuid
  );
```

サービスロール（バックエンド）はRLSをバイパスできるが、**ユーザー向けAPIでは必ずJWTのcompany_idを使用する**。RLSをバイパスするコードはセキュリティレビュー必須とする。

スキーマ管理

- `db/schema.sql`: 初期スキーマ（ベーステーブル定義）
- `db/migrations/{番号}_{説明}.sql`: 変更差分のみ（追加のみ。既存ファイルの変更禁止）
- マイグレーションはSupabase CLIまたは Supabase ダッシュボードのSQL Editorで適用

スケールアウト計画

フェーズ	規模	DB戦略
Phase 0-1	～100社	Supabase Pro
Phase 2-3	～3,000社	Supabase Enterprise / Team
Phase 4+	3,000社超	GCP Cloud SQL + カスタム認証移行を評価

移行トリガー: 月次インフラコスト ¥500万超 or Supabase SLAが要件を満たさなくなった時点。

影響

ポジティブな影響

- **開発速度**: 認証・RLS・ストレージが全て組み込みのため、セキュリティ基盤の実装工数を大幅削減（推定2週間→2日）
- **セキュリティ強度**: PostgreSQLのネイティブRLSはアプリケーション層フィルタリングより安全。コードバグによるテナント間データ漏洩のリスクが構造的に低い
- **pgvectorの活用**: ベクトル検索とリレーショナルデータを同一クエリで結合できる。外部ベクトルDBとの同期コストがゼロ
- **コスト予測可能性**: Supabase Proは固定 $25/月〜。スタートアップ初期の予算管理が容易

負の影響・トレードオフ

- **Supabaseロックイン**: Auth スキーマ（`auth.users`）はSupabase固有。移行時にユーザーデータの移行スクリプトが必要
- 緩和策: ユーザー情報は `public.users` テーブルにミラーリングし、`auth.users` への直接依存を最小化する
- **pgvectorのパフォーマンス制限**: 100万件以上のベクトルではHNSWインデックスのメモリ使用量が問題になる可能性
- 緩和策: テナント（company_id）ごとにパーティショニングし、クエリ時に `company_id` でフィルタリング後にベクトル検索
- **Realtimeの接続数制限**: Supabase Proのデフォルト接続数制限に注意。大規模同時接続はプラン変更が必要

今後の制約

1. **全テーブルにRLS必須**: RLSなしのテーブルはレビューでブロックする
2. **`company_id` フィルタ**: DB操作は必ず `company_id` でフィルタリングすること。クロステナントクエリは監査ログ記録必須
3. **マイグレーションは追加のみ**: 既存マイグレーションファイルの変更を禁止。修正は必ず新しいマイグレーションファイルで行う
4. **サービスロール使用制限**: `SUPABASE_SERVICE_ROLE_KEY` を使用するコードはRLSをバイパスするため、使用箇所を最小化しレビュー必須

関連

- `shachotwo/02_プロダクト設計.md` — Section 5: DB設計（テーブル定義）

- `db/schema.sql` — 初期スキーマ実装
- `db/migrations/` — マイグレーションファイル
- `db/supabase.py` — Supabaseクライアント実装
- ADR-001: Gemini LLM採用（LLMのレスポンスキャッシュをSupabaseに格納）
- ADR-003: LangGraph採用（PostgresSaverチェックポインターをSupabaseで実現）
- ADR-004: Voyage AI採用（埋め込みベクトルの格納先がSupabase pgvector）